

[データ構造](#) [区間クエリ](#) [矩形クエリ](#) [Wavelet Matrix](#) [Binary Trie](#) [セグメント木](#)
[Fenwick 木](#) [座標圧縮](#)

Wavelet Matrix（発展）

[解説動画はこちら](#)です。

1. 概要

Wavelet Matrix の基礎的な理解や実装方法については、[Wavelet Matrix（基礎）](#) で解説しました。

本講座では改めて、Wavelet Matrix を使うとどのような処理をできるのかについて、具体的な例をいくつか確認します。

なおほとんどの例について、[Wavelet Matrix（基礎）](#) を理解していれば、ひとつひとつの解法や実装方法に難しいところはないはずです。実装例の詳細は扱っていませんので、自身で実装して丁寧に確認してみてください。

2. 前提知識

[Wavelet Matrix（基礎）](#) の理解を前提とします。

また、Binary Trie, セグメント木, Fenwick 木などのデータ構造の知識を仮定します。

3. 値に対する XOR 操作

非負整数に対するビット単位 XOR 演算を $a \oplus b$ と書くことにします。

【問題 1】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R , $0 \leq k < R - L$ を満たす整数 k , 0 以上 $2^H - 1$ 以下の整数 x が与えられるので, $A_i \oplus x$ ($L \leq i < R$) の中で (0-based index で) 昇順 k 番目の要素の値を答えてください。

この問題は, $O((N + Q)H)$ 時間で解くことができます。

この問題では $x = 0$ の場合が Wavelet Matrix (基礎) **【問題 1】** です。 x を XOR するという要素が加わりましたが, 考え方は同じです。

Wavelet Matrix (基礎) **【問題 1】** では, 区間 $[L, R)$ の Binary Trie を考えて, 左部分木・右部分木の要素数を参照しながら適切な部分木に潜っていくという手順で解を求めました。

すべての A_i に x が XOR された場合にも同様のことが可能です。 $A_i \oplus x$ の 2^h の位は, x の 2^h の位が 0, 1 のどちらであるかに応じて A_i の 2^h の位そのまま, あるいは反転したものです。したがって, x の 2^h の位に応じてどちらの部分木を左部分木・右部分木と見なすかを入れ替えながら同様の手順をとればよいです。

4. 総和などの集約値

【問題 2】

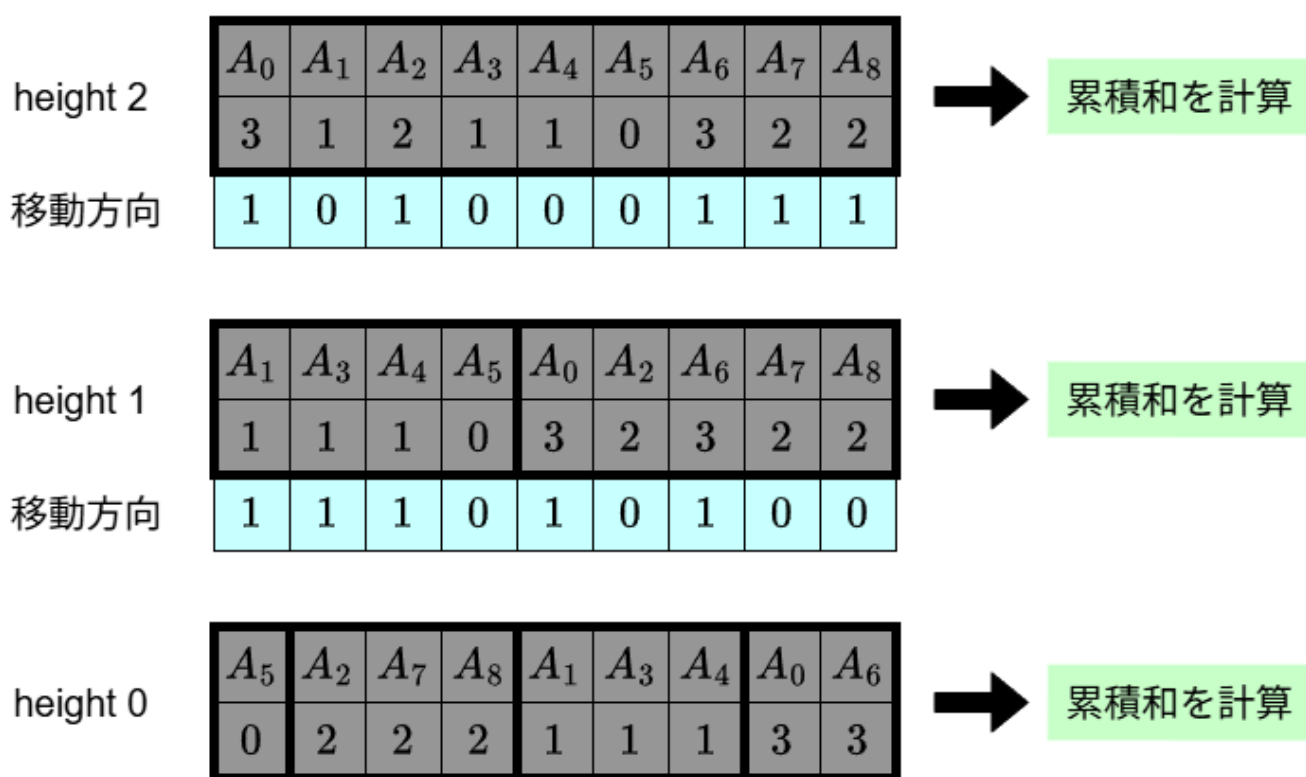
0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および, $0 \leq a < b \leq 2^H$ を満たす整数 a, b が与えられるので, $L \leq i < R$ かつ $a \leq A_i < b$ を満たす i に対する A_i の総和を求めてください。

求めるものが個数であるならば, Wavelet Matrix (基礎) 【問題 2】 で扱いました. その場合の解法はまず, $a \leq A_i < b$ という条件を $A_i < b$ と $A_i < a$ の差分として表すことで, $A_i < a$ という条件の場合の計算に帰着します. そして, 区間 $[L, R)$ の Binary Trie において $O(H)$ 個のノードの要素の**個数**の総和を求めればよいです.

今回は求めたい値が**総和**なので, $[L, R)$ の Binary Trie の各ノードの総和が求めればよいということになります.

Wavelet Matrix では, $[L, R)$ の Binary Trie のノードは, 3 つ組 (h, l, r) として表され, 高さ h における列の区間 $[l, r)$ に対応しました. このような区間での総和を求めるには, 各 h について累積和を計算しておけばよいです.



したがって本問題を解くためには, Wavelet Matrix の構築において,

- 各高さ $h + 1 \rightarrow h$ での移動方向を表す 0, 1 列の累積和
- 各高さ h での列の累積和

を求めておくというように, Wavelet Matrix を拡張すればよいです. 結局この問題も $O((N + Q)H)$ 時間で解くことができます.

【問題 3】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。さらに整数列 $X = (X_0, X_1, \dots, X_{N-1})$ が与えられます。

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および、 $0 \leq a < b \leq 2^H$ を満たす整数 a, b が与えられるので、 $L \leq i < R$ かつ $a \leq A_i < b$ を満たす i に対する X_i の総和を求めてください。

【問題 2】 では A_i の総和を求めていましたが、足すものは A_i である必然性は全くありません。 X_i を足すという本問題でも、解法は全く同じです。各高さごとに、 X を並べ替えたものの累積和を計算しておけばよいです。

なおより一般化すれば、 X は可換群の要素としても同じ解法が可能です。

【問題 4】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。さらに整数列 $X = (X_0, X_1, \dots, X_{N-1})$ が与えられます。

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および、 $0 \leq a < b \leq 2^H$ を満たす整数 a, b が与えられるので、 $L \leq i < R$ かつ $a \leq A_i < b$ を満たす i に対する X_i の $\max$ を求めてください（そのような i が存在しなければ -1 を出力してください）。

なお、 $\max$ の部分はそのまま可換モノイドに一般化できるので、代数構造の用語に慣れている人はそのように考えても良いでしょう。

これまでの説明では、 $a \leq A_i < b$ という条件を $A_i < a$ と $A_i < b$ の差によって処理してきましたが、これができない問題設定になっています。

しかしこの場合でも、求める値は「 $O(H)$ 個のノード (h, l, r) での X_i の $\max$ の $\max$ をとったもの」となります。

▶ 補足説明

結局、 $O(H)$ 回の区間 $\max$ クエリに帰着されます。この場合には各 h に対して、列に対するセグメント木を構築しておけば、時間計算量 $O((N + Q \log N)H)$ 時間でこの問題を解く

ことができます.

列が静的なので、セグメント木ではなくスパーステーブルや Sqrt Tree を使うことなども考えられるでしょう. ただし, 使用する空間の大きさにも注意してください. 例えば各 h に対してスパーステーブルを構築すると, 空間 $\Theta(NH \log N)$ が必要となります.

【問題 5】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます. さらに整数列 $X = (X_0, X_1, \dots, X_{N-1})$ が与えられます.

Q 個のクエリに答えてください. クエリは次の 2 種類です.

1. $0 \leq L < R \leq N$ を満たす整数 L, R および, $0 \leq a < b \leq 2^H$ を満たす整数 a, b が与えられるので, $L \leq i < R$ かつ $a \leq A_i < b$ を満たす i に対する X_i の $\max$ を求めてください.
2. $0 \leq i < N$ を満たす整数 i および, 整数 x が与えられるので, X_i を x に変更してください.

タイプ 2 のように X_i を点変更するクエリが加わりましたが, 解法には全く変化がありません. 各 h ごとにセグメント木を構築しておけば, 点変更もそのまま行えます.

$\max$ ではなく総和 (などの可換群) ならば, セグメント木の代わりに Fenwick 木を用いることもできます. このように, 用途に応じて補助的に使うデータ構造を差し替えることで, Wavelet Matrix の応用の柔軟性が広がります.

なおここでは点変更のクエリといっても, 変更対象は X_i であり, A_i は変更していないことに注意してください. 平衡二分木を使って A_i の点変更などに対応することも可能ですが, 実装はこれまでのものと比べて難しくなります.

5. 総和に関する二分探索

【問題 6】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます.

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および、非負整数 S が与えられるので、 $L \leq i < R$ かつ $0 \leq A_i < x$ を満たす i に対する A_i の総和が S 以下となるような 2^H 以下の最大の非負整数 x を求めてください。

「個数が S 以下」というような条件であれば、Wavelet Matrix (基礎) **【問題 1】** と同じです。これは個数ではなく総和に対する二分探索ですが、Wavelet Matrix において累積和テーブルを構築しておき各ノード (h, l, r) の総和を求められるようにしておけば、やはり $O((N + Q)H)$ 時間で解くことができます。

【問題 7】

0 以上 $2^H - 1$ 以下の整数からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます。

Q 個のクエリに答えてください。クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R および、非負整数 S が与えられるので、 $\sum_{L \leq i < R} \max(x - A_i, 0) \leq S$ を満たすような最大の非負整数 x を求めてください。

左辺は x について単調増加なので、やはり二分探索が使えます。

値の区間 $[c, d)$ に対応するノードを調べているときに、 $[0, c)$ 部分の個数と総和を保持するようにしながら部分木を潜っていくことで、 $O((N + Q)H)$ 時間で解くことができます。

6.2 次元点群に対する矩形処理

【問題 8】

座標平面上の N 個の点 (A_i, B_i) ($0 \leq i < N$) が与えられます。 Q 個のクエリに答えてください。

クエリでは整数 L, R, D, U が与えられるので、 $L \leq A_i < R$ かつ $D \leq B_i < U$ を満たす i の個数を求めてください。

これまでは静的な非負整数列 $(A_0, A_1, \dots, A_{N-1})$ という設定でしたが、この問題は 2 次元点群に対する問題です。

とはいえ競技プログラミングに慣れている方ならば、この問題が静的な非負整数列の問題に帰着できることはすぐに分かるでしょう。

まず A_i についてソートして、 A_i が単調増加になるようにインデックスをつけなおします。 $L \leq A_i < R$ という条件は、添字の区間の条件に書き直すことができます。

B_i についても座標圧縮して同じようにすれば、与えられる点の y 座標は 0 以上 $N - 1$ 以下の整数である場合に帰着できます。これは高さが $\lceil \log_2 N \rceil$ の Wavelet Matrix で扱えます。

結局、この問題は $O((N + Q) \log N)$ 時間で解くことができます。

【問題 9】

整数の 3 つ組 (A, B, X) が N 個与えられます。

Q 個のクエリに答えてください。クエリは次の 3 種類です。

1. 新たに 3 つ組 (A, B, X) を追加する。
2. 既に存在する 3 つ組 (A, B, X) を削除する。
3. 整数 L, R, D, U が与えられるので、 $L \leq A < R$ かつ $D \leq B < U$ を満たす 3 つ組 (A, B, X) に対する X の max を求めてください。

点の追加・削除のクエリがありますが、このような場合にははじめにクエリをすべて先読みしてしまい、問題を通してある時刻に存在する点 (A_i, B_i) を列挙してしまいます。すると、点の追加・削除は「 X の値を反映させる」「 X の値を単位元に変更する」という値の変更として表すことができます。

あとは座標圧縮すれば、【問題 5】と同様に $O((N + Q) \log^2(N + Q))$ 時間で解くことができます。

7. 関連問題

1. https://atcoder.jp/contests/abc339/tasks/abc339_g

2. <https://yukicoder.me/problems/no/2065>
3. <https://yukicoder.me/problems/no/924>
4. https://judge.yosupo.jp/problem/rectangle_sum
5. https://judge.yosupo.jp/problem/point_add_rectangle_sum
6. https://atcoder.jp/contests/abc266/tasks/abc266_h
7. https://atcoder.jp/contests/joiopen2025/tasks/joiopen2025_b

▶ 問題について

8. まとめ

本講座では、Wavelet Matrix のさまざまな利用方法について確認しました。

Wavelet Matrix は、競技プログラミングで用いられるデータ構造の中ではそれほど知名度が高いわけではありません（このことは歴史的にも比較的新しいデータ構造であることも影響していると思います）。しかし、簡潔な実装でかなり広い応用範囲を持っていることが確認できたと思います。

9. 参考文献

Wavelet Matrix（基礎） と同一の参考文献を再掲しています。

1. Roberto Grossi, Ankur Gupta, Jeffrey Scott Vitter. High-Order Entropy-Compressed Text Indexes. Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). pp. 841–850. 2003.
<https://pages.di.unipi.it/grossi/PAPERS/sodaconf03FINAL-LATEST.pdf>.
2. Francisco Claude, Gonzalo Navarro, Alberto Ordóñez Pereira. The wavelet matrix: An efficient wavelet tree for large alphabets. Information Systems. Vol. 47. pp. 15–32. 2015. DOI: 10.1016/j.is.2014.06.002.
<https://doi.org/10.1016/j.is.2014.06.002>. (Preprint)
<https://arxiv.org/abs/1405.1220>.
3. USACO Guide. Wavelet Tree. <https://usaco.guide/adv/wavelet>. (閲覧日: 2026-03-04).

4. Preferred Networks Tech Blog. ウェーブレット木の世界.
https://tech.preferred.jp/ja/blog/wavelettree_world/. (閲覧日: 2026-03-04).
5. Mitl_7. ウェーブレット行列 (wavelet matrix). <https://mitl-7.hatenablog.com/entry/2018/04/28/152259>. (閲覧日: 2026-03-04).
6. Robinson Castro, Nico Lehmann, Jorge Pérez, Bernardo Subercaseaux.
Wavelet Trees for Competitive Programming. Olympiads in Informatics. Vol. 10. pp. 19–37. 2016. DOI: 10.15388/ioi.2016.02.
<https://doi.org/10.15388/ioi.2016.02>.
https://ioinformatics.org/journal/v10_2016_19_37.pdf. (閲覧日: 2026-03-04).
7. Wikipedia. 簡潔データ構造. <https://ja.wikipedia.org/wiki/簡潔データ構造>. (閲覧日: 2026-04-16).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)